

Projet OCR

liam.corbin

thibault.daguier

matthieu.oursin

gustave.molina

October 2025

1 Introduction

Dans ce rapport, nous allons vous présenter dans un premier temps les rôles et les missions de chaque membre du groupe, ce qu'il a fait pour le groupe, ce qu'il devait faire pour cette soutenance et quelles tâches il a réalisées.

Ensuite, dans un deuxième temps, nous vous parlerons des objectifs attendus lors de cette soutenance, de la répartition des tâches entre les membres du groupe pour réussir à atteindre ces objectifs, et si nous avons réussi à les atteindre, c'est-à-dire l'avancement réel du projet ainsi que les éventuels retards ou avancées que nous avons réalisés.

Enfin, dans un dernier temps, nous vous expliquerons, pour chaque fonctionnalité attendue, les choix technologiques réalisés, comme le choix des bibliothèques utilisées, les difficultés rencontrées et les solutions que nous avons trouvées pour résoudre ces problèmes et permettre le bon fonctionnement de

chaque fonctionnalité. Nous détaillerons aussi les différents algorithmes utilisés et que nous avons dû appliquer pour chaque fonctionnalité.

En conclusion, ce rapport vous donnera une vue d'ensemble du projet et des fonctionnalités réalisées.

Table des matières

1	Introduction	1
2	Rôle et mission de chaque membre	4
3	Objectif, planification des tâches et avancement réel du projet	5
3.1	Objectifs	5
3.2	Répartition des tâches	6
3.3	Avancement du projet	7
4	Difficultés éventuelles, choix technologiques et algorithmes	8
4.1	Chargement de l'image, rotation, suppression des couleurs . .	8
4.2	Algorithme de résolution de la grille	9
4.3	Preuve de concept du réseau de neurones	10
4.3.0.1	Principe de la fonction NXOR :	10
4.3.0.2	Structure du réseau :	10
4.3.0.3	Algorithme utilisé :	11
4.3.0.4	Entraînement :	11
4.3.0.5	Résultats et interprétation :	12
4.3.0.6	Choix technologiques et difficultés rencontrées :	12
4.3.0.7	Conclusion :	13
4.4	Détection de la grille et des éléments sur l'image	13

2 Rôle et mission de chaque membre

Le rôle de Matthieu Oursin en ce début de projet a été de créer le programme *solver* demandé pour cette soutenance, ainsi que de développer le programme qui chargera l'image sans les couleurs et avec une rotation manuelle.

Thibault Daguier et Liam Corbin ont été chargé en ce début de projet de la création du réseau de neurones, ici que la porte logique demandée.

Gustave Molina s'est chargé de la détection de la grille et de la liste de mots ainsi que le découpage des lettres de la grille.

3 Objectif, planification des tâches et avancement réel du projet

3.1 Objectifs

Les objectifs de cette première soutenance étaient clairs :

- Chargement d'une image et suppression des couleurs ;
- Rotation manuelle de l'image ;

Détection de la position :

- de la grille ;
- de la liste de mots ;
- des lettres dans la grille ;
- des mots de la liste ;
- des lettres dans les mots de la liste.

- Découpage de l'image (sauvegarde de chaque lettre sous la forme d'une image) ;

- Implémentation de l'algorithme de résolution d'une grille de mots cachés. Vous devrez implémenter cet algorithme dans le programme en ligne de commande *solver* ;

- Une preuve de concept de votre réseau de neurones. Pour cette preuve, vous réaliserez un mini-réseau capable d'apprendre la fonction logique $A.B + A.B$ (A et B étant des variables booléennes n'acceptant que les valeurs 0 et 1).

3.2 Répartition des tâches

Pour les tâches, nous nous les sommes réparties de cette façon :

- Liam Corbin s’est occupé de la preuve de concept du réseau de neurones et la détection de la liste de mots.
- Gustave Molina s’est occupé de la détection de la position de la grille, de la liste de mots, des lettres dans la grille, des mots de la liste, des lettres dans les mots de la liste et du découpage de l’image.
- Thibault Daguiet a travaillé avec Liam Corbin sur la preuve de concept du réseau de neurones. Étant donné qu’il s’agissait de quelque chose de nouveau, nous avons préféré mettre deux membres du groupe sur cette tâche.
- Matthieu Oursin s’est occupé du chargement d’une image et de la suppression de ses couleurs, de la rotation manuelle de l’image et de l’implémentation de l’algorithme de résolution de la grille de mots cachés.

3.3 Avancement du projet

Pour la partie du chargement de l'image ainsi que la suppression des couleurs et de la rotation manuelle, l'objectif a été atteint. Toutes ces fonctionnalités sont implémentées et fonctionnent parfaitement. Le programme *Solver*, chargé de la résolution de la grille, renvoie bien les coordonnées de la première et de la dernière lettre si le mot est présent dans la grille, et « Not Found » sinon.

Avec le programme `image.c`, qui possède trois paramètres, on peut entrer en paramètre le fichier image à charger, le fichier image dans lequel sauvegarder, ainsi que le nombre de rotations à effectuer dans le sens antihoraire. Avec ces trois paramètres, apparaît alors sur l'écran l'image sans couleurs, avec une rotation si nécessaire.

En conclusion, ces objectifs ont été réalisés.

Pour la partie de détection et de découpage de la grille. Le fichier « `detection.c` » a été développé afin de charger une image en entrée, identifier la zone correspondant à la grille, puis découper celle-ci en sous-images représentant chaque case.

La fonction principale, « `detect-and-cut` », utilise « `stb-image.h` » pour le chargement des pixel et la récupération des dimensions de l'image. Le programme procède ensuite à un découpage régulier de la grille selon un nombre défini de lignes et de colonnes, calculé à partir de la taille totale de l'image. Chaque case extraite est sauvegardée en tant que fichier dans un dossier créé préalablement. Le module est actuellement opérationnel pour les images propres (niveau 1 du cahier des charges), c'est-à-dire sans rotation. La détection automatique des bordures de la grille n'est pas encore implémentée : le programme suppose pour l'instant que la grille occupe l'ensemble de l'image.

4 Difficultés éventuelles, choix technologiques et algorithmes

4.1 Chargement de l'image, rotation, suppression des couleurs

Pour le chargement de l'image, nous avons dû passer par une bibliothèque à cause du format de l'image. En effet, les images peuvent être au format `.jpg`, `.png` ou encore `.jpeg`, et ces formats possèdent chacun leurs propres spécifications et problèmes à gérer.

Par exemple, pour gérer uniquement le format PNG, il aurait fallu créer un décodeur capable de décoder l'algorithme de compression lié à ce format, ce qui reviendrait à recréer un projet complexe de centaines, voire de milliers de lignes.

Nous nous sommes donc tournés vers une bibliothèque qui gèrait ces problèmes à notre place : **GTK3**, une bibliothèque robuste et reconnue dans le domaine de l'affichage graphique.

Celle-ci nous permet de réaliser un affichage graphique moderne, mais aussi de charger et de modifier des images.

Grâce à cette bibliothèque, nous avons pu réaliser le chargement de l'image et la suppression des couleurs.

Pour la rotation selon un angle, il n'y avait pas de possibilité directe avec GTK3. Nous avons donc dû utiliser une sous-bibliothèque de GTK3 : **Cairo**. Cette sous-bibliothèque permet de créer des surfaces dessinables et modifiables à souhait, plus librement que GTK, puis de les sauvegarder avec GTK3.

Nous avons donc créé une fonction `rotate_pixbuf` qui effectue une rotation d'un certain angle (entré en paramètre) sur un *pixbuf*, c'est-à-dire un ensemble de pixels propre à GTK3, et qui renvoie ce *pixbuf* avec la rotation appliquée.

Nous avons ensuite simplement sauvegardé et affiché l'image grâce à GTK3, ce qui se fait plutôt simplement.

Dans cette partie, nous n'avons donc pas rencontré beaucoup de problèmes, si ce n'est les calculs liés à la rotation de l'image via Cairo, qui étaient un peu techniques et ont demandé des recherches. La recherche de chaque fonction liée aux bibliothèques sur leurs wikis respectifs était également un peu fastidieuse.

4.2 Algorithme de résolution de la grille

Pour l'algorithme de résolution de la grille, nous avons dû convertir l'image en matrice pour la parcourir. Cela s'est fait à l'aide d'une fonction permettant de lire le fichier et de recopier chaque élément de chaque ligne dans la matrice correspondante.

Ensuite, nous avons parcouru la matrice jusqu'à rencontrer la première lettre du mot. Une fois trouvée, nous regardons alors dans les huit directions possibles si nous trouvons les lettres suivantes ; sinon, nous continuons le parcours de la matrice.

La partie où il faut examiner les huit directions a été un peu difficile, car il fallait penser à tous les cas possibles et aussi mémoriser la position de la première lettre. En effet, s'il y avait autour de cette lettre plusieurs occurrences de la seconde lettre du mot, il fallait gérer toutes ces éventualités, ce

qui a été un peu compliqué. Au final, le programme fonctionne bien.

En conclusion, sur cette partie, l'algorithme a été un peu difficile à concevoir, mais une fois chaque cas bien géré et la structure de base mise en place, il n'y a pas eu de grandes difficultés dans la création de cet algorithme.

4.3 Preuve de concept du réseau de neurones

Pour cette première soutenance, la tâche confiée à Liam Corbin et Thibault Daguié était de réaliser une preuve de concept démontrant le bon fonctionnement d'un réseau de neurones artificiel (RNA) en langage C. Ce réseau devait être capable d'apprendre une fonction logique non linéaire : la fonction NXOR (équivalente à $A \cdot B + \overline{A} \cdot \overline{B}$).

L'objectif principal était de vérifier que notre implémentation du perceptron multicouche, de la propagation avant et de la rétropropagation du gradient fonctionnait correctement avant de passer à la reconnaissance de caractères à partir d'images.

4.3.0.1 Principe de la fonction NXOR :

La fonction NXOR renvoie la valeur 1 lorsque les deux entrées sont identiques, et 0 dans le cas contraire. C'est une fonction classique utilisée pour tester la capacité d'un réseau de neurones à apprendre une relation non linéaire, car un perceptron simple ne peut pas la modéliser correctement.

4.3.0.2 Structure du réseau :

Le réseau conçu est un petit perceptron multicouche (MLP) composé de :

- 2 neurones en entrée (les variables booléennes A et B) ;

- 2 neurones dans la couche cachée avec fonction d'activation sigmoïde ;
- 1 neurone de sortie, également avec fonction d'activation sigmoïde.

Les poids et biais sont initialisés aléatoirement dans l'intervalle $[-1, 1]$ à l'aide d'une fonction de génération pseudo-aléatoire.

4.3.0.3 Algorithme utilisé :

L'apprentissage est réalisé par **rétropropagation du gradient**. Pour chaque itération (ou *epoch*), le réseau effectue une propagation avant, calcule l'erreur quadratique entre la sortie attendue et la sortie obtenue, puis met à jour les poids selon la règle :

$$w \leftarrow w - \eta \frac{\partial E}{\partial w}$$

où η est le taux d'apprentissage et E l'erreur globale du réseau.

Un taux d'apprentissage de 0.1 a été utilisé, sur environ 10 000 itérations, ce qui permet une bonne convergence vers la fonction cible.

4.3.0.4 Entraînement :

Le réseau a été entraîné sur les quatre couples d'entrée possibles :

$$(0, 0) \rightarrow 1, \quad (0, 1) \rightarrow 0, \quad (1, 0) \rightarrow 0, \quad (1, 1) \rightarrow 1$$

Après plusieurs milliers d'itérations, le réseau converge et donne les résultats suivants :

Entrée A	Entrée B	Sortie du réseau	Sortie attendue
0	0	0.97	1
0	1	0.03	0
1	0	0.04	0
1	1	0.96	1

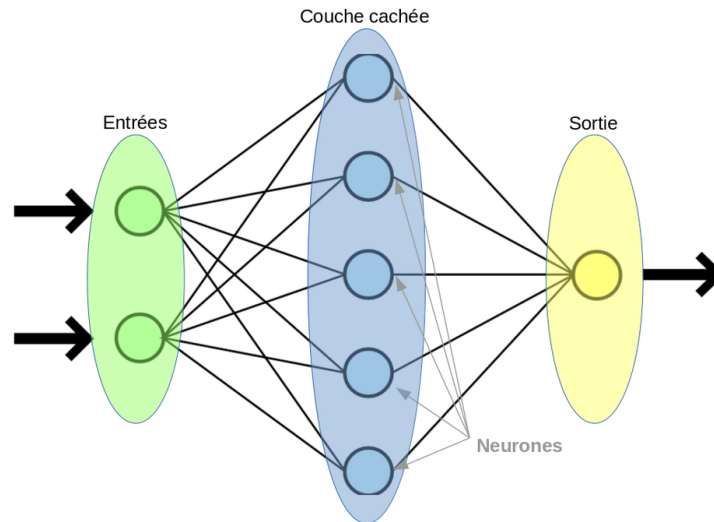


FIGURE 1 – Réseau de neurones

4.3.0.5 Résultats et interprétation :

Les sorties approchent très bien les valeurs attendues, prouvant que le réseau est capable de modéliser une fonction non linéaire grâce à la présence d'une couche cachée. Cette réussite valide la mise en œuvre correcte des trois composantes principales :

- la propagation avant (calcul des sorties intermédiaires) ;
- la rétropropagation de l'erreur ;
- la mise à jour progressive des poids.

4.3.0.6 Choix technologiques et difficultés rencontrées :

L'implémentation a été réalisée intégralement en **langage C**, sans bibliothèque externe, afin de garantir la compatibilité avec le reste du projet et une compréhension fine du fonctionnement interne du réseau.

Les principales difficultés ont concerné :

- la gestion de la mémoire dynamique pour stocker les poids sous forme de tableaux à deux dimensions ;

- la vérification du bon calcul des gradients, notamment le signe de la dérivée lors de la rétropropagation ;
- la lenteur de convergence lorsque le taux d'apprentissage est mal ajusté.

4.3.0.7 Conclusion :

Cette preuve de concept a permis de valider notre implémentation du réseau de neurones et de nous familiariser avec la manipulation des structures et des calculs nécessaires à l'apprentissage automatique. Elle constitue une base solide pour la suite du projet, où ce réseau sera étendu à la reconnaissance de caractères issus du découpage d'image.

4.4 Détection de la grille et des éléments sur l'image

Choix technologiques

La bibliothèque "stb-image.h" a été utilisée pour le chargement et la manipulation des images, car elle permet une intégration directe sans dépendances externes, tout en prenant en charge les formats standards.

Le traitement des images est effectué à partir d'un tableau de pixels chargé en mémoire sous forme d'un tableau 2D. Cette approche a permis un contrôle précis et simplifie la logique du découpage. Le choix d'un découpage déterministe, basé sur les dimensions connues de la grille, permet des résultats constants et limite la complexité algorithmique.

Algorithme de détection de la grille

Le programme se compose principalement d'une fonction, « detect-and-cut », qui effectue les étapes suivantes :

1. Chargement de l'image à l'aide de "stb-image", avec récupération de la largeur, de la hauteur et du nombre de canaux de couleur.

2. Calcul du pas de découpage, défini comme la largeur et la hauteur d'une case, obtenus en divisant les dimensions de l'image par le nombre de lignes et de colonnes.
3. Extraction des sous-images à partir de boucles imbriquées parcourant la grille, chaque case étant sauvegardée sous forme d'image indépendante.

Difficultés rencontrées

Plusieurs difficultés ont été rencontrées au cours du développement :

1. Alignement des cases : un décalage de quelques pixels dans le calcul du pas pouvait entraîner des erreurs visibles sur les sous-images extraites.
2. Absence de détection automatique de la grille : à ce stade, le découpage suppose que la grille occupe la totalité de l'image. Cette simplification fonctionne pour les tests de niveau 1, mais devra être améliorée pour les niveaux supérieurs.